SP25-BL-ENGR-E516-12138

ShieldScrape: Secure & Scalable Cloud-Based Web Data Collection

Team Members

- **Dhuru, Aryan Mayur** - (adhuru@iu.edu) -**Cloud Infrastructure & Security**: Manages AWS architecture, security policies, and fault tolerance mechanisms.
- **Ambol, Suhana** - (sambol@iu.edu) - **Dashboard & Visualization**: Builds the frontend dashboard with AWS Amplify and analytics with QuickSight.
- **Mate, Shivali** - (shimate@iu.edu) - **Web Scraping & API Integration**: Develops scraping logic with Scrapy/Puppeteer and integrates with API Gateway.
- **Sonawane, Varunkumar** - (vsonawa@iu.edu) - **Backend & Data Processing**: Oversees ETL pipeline, AWS Glue, and data transformation logic.

Submission Date: March 09, 2025
Course: ENGR-E516 Engineering Cloud Computing
Institution: Indiana University Bloomington, Luddy School of Informatics, Computing, and Engineering

Problem Statement

Web data collection is indispensable for industries like e-commerce, journalism, and academic research, enabling tasks such as price monitoring, news aggregation, and sentiment analysis. However, traditional web scraping tools face significant challenges:

- **Scalability**: Single-threaded or local scrapers struggle with large-scale data extraction from multiple domains.
- **Security**: Lack of encryption and IP rotation increases vulnerability to data breaches and anti-scraping measures (e.g., CAPTCHAs, IP bans).
- **Efficiency**: Sequential processing and manual workflows limit throughput and real-time applicability.
- **Resilience**: Single-point failures and lack of fault tolerance disrupt operations.

Existing solutions like Scrapy or BeautifulSoup excel at scraping but lack native cloud integration for distributed execution, automation, and advanced security. Most of the recent cloud-based projects showcase scalable data pipelines using AWS services, but they focus on post-collection analysis rather than secure, parallel web scraping.

ShieldScrape addresses these gaps by building a fully cloud-native, distributed web scraping pipeline. It leverages AWS's ecosystem for parallel computing, automated workflows, and robust security, ensuring scalability, efficiency, and resilience while minimizing detection risks from target websites.

Project Goals & Expected Results

Primary Objectives:

1. **Distributed Web Scraping**: Deploy a system using AWS Lambda (serverless) and AWS Fargate (containerized) with API Gateway to scrape data from 100+ websites in parallel.
2. **Automated Data Pipeline**: Use Apache Airflow (on AWS ECS) and AWS Step Functions for orchestration, paired with AWS Glue for ETL to transform raw HTML into structured JSON/CSV.
3. **Advanced Security**: Implement AWS Secrets Manager for credential storage, AWS WAF for bot protection, IAM policies for access control, and AWS Shield for DDoS mitigation.
4. **Optimized Storage**: Store raw data in Amazon S3, metadata in Amazon DynamoDB, and structured data in Amazon RDS (PostgreSQL), with Amazon OpenSearch for full-text search.
5. **Real-Time Monitoring**: Integrate AWS CloudWatch for performance metrics and AWS QuickSight for a user-friendly dashboard displaying scraping stats and insights.
6. **High Availability**: Achieve fault tolerance with auto-scaling groups, Elastic Load Balancers (ELB), and multi-region S3 replication.

Expected Results:

- **Performance**: Scrape 100 webpages in under 5 minutes with 90% success rate across diverse domains.
- **Automation**: Fully automated pipeline with daily scheduled runs and error recovery.
- **Security**: Zero unauthorized access incidents and <5% detection rate by anti-scraping mechanisms.
- **Analytics**: Real-time dashboard showing scraping throughput, data volume, and error rates.
- **Resilience**: System uptime >99.9% with automatic failover under simulated failures.

Impact & Contributions:

- **Scalable Architecture**: A reusable blueprint for cloud-based scraping with parallel execution.
- **Security Innovation**: Advanced anti-detection and data protection mechanisms tailored for web scraping.

Methodology & Required Resources

Technology Stack:

Category	Technology/Service	Description
Web Scraping	AWS Lambda, AWS Fargate	Serverless and containerized scraping for parallel execution.
	API Gateway	Secure, rate-limited entry point for scraper invocation.
	Scrapy, Puppeteer	Scrapy for static sites; Puppeteer for dynamic JavaScript-rendered pages.
Workflow Orchestration	Apache Airflow (AWS ECS)	Schedules and manages scraping, ETL, and storage workflows via DAGs.
	AWS Step Functions	Coordinates microservices for complex workflows (e.g., error handling).
Data Processing	AWS Glue	Extracts, transforms, and loads data into structured formats.
	AWS Lambda	Lightweight post-processing tasks (e.g., data validation).
	Amazon Comprehend	Optional NLP for sentiment analysis or entity extraction from scraped content.
Storage & Databases	Amazon S3	Stores raw HTML and processed files with versioning and replication.
	Amazon DynamoDB	Key-value store for metadata (e.g., URL, timestamp, status).
	Amazon RDS (PostgreSQL)	Relational storage for structured data (e.g., tables of extracted fields).

	Amazon OpenSearch	Indexes scraped content for fast search and analytics.
Security & Compliance	AWS WAF	Protects against bot detection and malicious requests.
	AWS Secrets Manager	Securely stores API keys, proxy credentials, and database passwords.
	IAM Roles & Policies	Granular access control for all services.
	AWS Shield	Mitigates DDoS attacks during high traffic scraping.
Monitoring & Alerts	AWS CloudWatch	Logs metrics (e.g., Lambda execution time, S3 upload latency).
	AWS SNS	Sends alerts for failures or anomalies (e.g., scraping errors).
Visualization	AWS QuickSight	Dashboards for real-time scraping stats and data insights.
	AWS Amplify (React)	Hosts a minimal frontend for URL input and result display.

Implementation Steps:

1. **Infrastructure Setup (Weeks 1-2):**
    - Provision AWS resources: Lambda, Fargate, ECS cluster, S3 buckets, RDS, and DynamoDB.
    - Configure API Gateway with throttling and WAF rules to mimic legitimate traffic.
    - Set up CloudWatch dashboards and SNS for monitoring and alerts.
    - Implement CI/CD pipelines using AWS CodePipeline and AWS CodeBuild for deployment automation.
2. **Scraping Development (Weeks 2-3):**
    - Build Scrapy spiders for static sites and Puppeteer scripts for dynamic sites.
    - Deploy spiders to Lambda (serverless, lightweight tasks) and Fargate (heavier, persistent tasks).
    - Implement proxy rotation via API Gateway and third-party services (e.g., Bright Data).
3. **Data Pipeline (Weeks 3-4):**
    - Use AWS Glue to crawl S3, extract key fields (e.g., title, price), and output to RDS/DynamoDB.
    - Add Amazon Comprehend for optional NLP (e.g., sentiment scoring of reviews).
    - Index processed data in OpenSearch for searchability.
4. **Security Integration (Weeks 4-5):**
    - Encrypt S3 data with KMS and secure RDS/DynamoDB with IAM roles.
    - Configure WAF to block suspicious patterns and Shield for DDoS protection.
    - Store credentials in Secrets Manager and test access controls.
5. **Automation (Weeks 5-6):**
    - Deploy Airflow on ECS with DAGs for daily scraping, ETL, and storage updates.
    - Use Step Functions for retry logic on failed scrapes or ETL jobs.

6. **Frontend & Analytics (Weeks 6-7):**
    - Build a React app with Amplify for URL submission and result viewing.
    - Create QuickSight dashboards visualizing scraping success rate, data volume, and trends.
7. **Resilience & Testing (Weeks 7-8):**
    - Add auto-scaling to Fargate and ELB for Lambda traffic distribution.
    - Test fault tolerance with simulated failures (e.g., S3 outage, Lambda timeout).
    - Optimize performance under high load (e.g., 500 concurrent requests).

Required Resources:

- **AWS Free Tier**: Lambda (1M requests/month), S3 (5GB), RDS (20GB), DynamoDB (25GB).
- **Jetstream2 Cloud**: Backup testing environment if AWS credits are exhausted.
- **Software**: Python 3.9, Scrapy 2.11, Puppeteer (Node.js), Boto3 (AWS SDK), Airflow 2.7.
- **Hardware**: Local machines for development; AWS for production.
- **Budget**: Aim for < $10/month beyond free tier using cost-efficient services.

Project Timeline (8 Weeks)

Week	Tasks	Deliverables
1	Research, design AWS architecture, set up initial Lambda/Fargate	Architecture diagram, test Lambda
2	Develop Scrapy/Puppeteer scripts, deploy to Lambda/Fargate, add API Gateway	Parallel scraping prototype
3	Configure S3, RDS, DynamoDB; implement Glue ETL	Midterm report with initial results
4	Integrate WAF, Secrets Manager, Shield; test security	Secure scraping pipeline
5	Deploy Airflow on ECS, define DAGs; add Step Functions	Automated workflow
6	Build Amplify frontend, set up QuickSight dashboards	User interface & analytics
7	Add auto-scaling, ELB, multi-region S3; test fault tolerance	Resilient system
8	Performance testing, debugging, finalize report & presentation	Final report, demo

Conclusion

ShieldScrape is a cloud-heavy, cutting-edge web scraping solution that leverages AWS's extensive service ecosystem for scalability, security, and automation. By integrating parallel scraping (Lambda/Fargate), automated pipelines (Airflow/Glue), and robust security (WAF/Shield), it showcases the power of cloud computing for real-time data collection. The project's deliverables, which are, a scalable system, detailed report, and live demo—will highlight its efficiency, resilience, and innovation, meeting E516's focus on cloud backend complexity and parallel processing.

Next Steps: Begin AWS account setup, provision initial resources, and prototype a Lambda-based scraper by Week 1's end.

References

[1] Scrapy Documentation: https://docs.scrapy.org/

[2] AWS CloudWatch: https://aws.amazon.com/cloudwatch/

[3] Apache Airflow: https://airflow.apache.org/

[4] AWS Security Best Practices: https://aws.amazon.com/whitepapers/aws-security-best-practices/

[5] Amazon Elastic Kubernetes Service (EKS) Documentation: https://docs.aws.amazon.com/eks/index.html

[6] Kubernetes.io: https://kubernetes.io/docs/

[7] CI/CD for Kubernetes with AWS: https://aws.amazon.com/blogs/containers/ci-cd-for-kubernetes-using-github-actions-and-amazon-eks/

[8] Implementing Microservices on AWS: https://aws.amazon.com/microservices/